Compendio de funciones

Raspberry Pi Pico SDK

Profesor: Saúl Villegas

Función / prototipo	Parámetros	Biblioteca	Descripción	Ejemplo de uso
`void gpio_init(uint gpio);`	gpio: número del pin GPIO que se desea inicializar.	pico/stdlib.h o hardware/gpio.h	Inicializa un GPIO para utilizarlo mediante SIO.	`gpio_init(25);`
`void gpio_set_dir(uint gpio, bool out);`	gpio: número del GPIO. out: true/GPIO_OUT para salida; false/GPIO_IN para entrada.	pico/stdlib.h o hardware/gpio.h	Configura la dirección de un GPIO.	`gpio_set_dir(25, GPIO_OUT);`
`void gpio_put(uint gpio, bool value);`	gpio: número del GPIO. value: nivel lógico 0 o 1.	pico/stdlib.h o hardware/gpio.h	Escribe un nivel lógico en una salida digital.	`gpio_put(25, 1);`
`bool gpio_get(uint gpio);`	gpio: número del GPIO que se desea leer.	pico/stdlib.h o hardware/gpio.h	Lee el nivel lógico presente en un GPIO.	`bool estado = gpio_get(15);`
`void gpio_pull_up(uint gpio);`	gpio: GPIO donde se activará la resistencia interna.	pico/stdlib.h o hardware/gpio.h	Activa la resistencia interna pull-up.	`gpio_pull_up(15);`
`void gpio_pull_down(uint gpio);`	gpio: GPIO donde se activará la resistencia interna.	pico/stdlib.h o hardware/gpio.h	Activa la resistencia interna pull-down.	`gpio_pull_down(15);`
`void gpio_disable_pulls(uint gpio);`	gpio: número del GPIO.	pico/stdlib.h o hardware/gpio.h	Desactiva las resistencias internas pull-up y pull-down.	`gpio_disable_pulls(15);`
`void gpio_set_function(uint gpio, gpio_function_t fn);`	gpio: pin. fn: función periférica asignada, por ejemplo GPIO_FUNC_UART o GPIO_FUNC_PWM.	pico/stdlib.h o hardware/gpio.h	Asigna al pin una función periférica alternativa.	`gpio_set_function(0, GPIO_FUNC_UART);`
`void sleep_ms(uint32_t ms);`	ms: tiempo de espera en milisegundos.	pico/stdlib.h o pico/time.h	Suspende la ejecución durante una cantidad de milisegundos.	`sleep_ms(500);`
`void sleep_us(uint64_t us);`	us: tiempo de espera en microsegundos.	pico/stdlib.h o pico/time.h	Suspende la ejecución durante una cantidad de microsegundos.	`sleep_us(100);`
`uint32_t time_us_32(void);`	No recibe parámetros.	pico/stdlib.h o pico/time.h	Obtiene los 32 bits bajos del contador de tiempo en microsegundos.	`uint32_t t = time_us_32();`
`uint64_t time_us_64(void);`	No recibe parámetros.	pico/stdlib.h o pico/time.h	Obtiene el tiempo transcurrido en microsegundos con 64 bits.	`uint64_t t = time_us_64();`
`void tight_loop_contents(void);`	No recibe parámetros.	pico/stdlib.h o pico/platform.h	Indica que el programa está intencionalmente dentro de un ciclo de espera.	`while (!gpio_get(15)) tight_loop_contents();`
`void adc_init(void);`	No recibe parámetros.	hardware/adc.h	Inicializa el periférico ADC.	`adc_init();`
`void adc_gpio_init(uint gpio);`	gpio: GPIO analógico que se desea utilizar.	hardware/adc.h	Configura un GPIO como entrada analógica.	`adc_gpio_init(26);`
`void adc_select_input(uint input);`	input: número de canal ADC.	hardware/adc.h	Selecciona el canal que será convertido.	`adc_select_input(0);`
`uint16_t adc_read(void);`	No recibe parámetros.	hardware/adc.h	Realiza una conversión ADC y devuelve el valor digital.	`uint16_t valor = adc_read();`
`void adc_set_temp_sensor_enabled(bool enable);`	enable: true para activar el sensor interno; false para desactivarlo.	hardware/adc.h	Habilita o deshabilita el sensor interno de temperatura.	`adc_set_temp_sensor_enabled(true);`
`uint pwm_gpio_to_slice_num(uint gpio);`	gpio: GPIO utilizado para PWM.	hardware/pwm.h	Obtiene el número de slice PWM asociado a un GPIO.	`uint slice = pwm_gpio_to_slice_num(16);`
`uint pwm_gpio_to_channel(uint gpio);`	gpio: GPIO utilizado para PWM.	hardware/pwm.h	Determina si el GPIO corresponde al canal A o B del slice.	`uint canal = pwm_gpio_to_channel(16);`
`void pwm_set_wrap(uint slice_num, uint16_t wrap);`	slice_num: slice PWM. wrap: valor máximo del contador.	hardware/pwm.h	Define el valor TOP del contador PWM.	`pwm_set_wrap(slice, 999);`
`void pwm_set_gpio_level(uint gpio, uint16_t level);`	gpio: pin PWM. level: nivel de comparación.	hardware/pwm.h	Establece directamente el nivel PWM correspondiente a un GPIO.	`pwm_set_gpio_level(16, 500);`
`void pwm_set_chan_level(uint slice_num, uint chan, uint16_t level);`	slice_num: slice. chan: PWM_CHAN_A o PWM_CHAN_B. level: nivel de comparación.	hardware/pwm.h	Configura el nivel de comparación de un canal PWM.	`pwm_set_chan_level(slice, PWM_CHAN_A, 500);`
`void pwm_set_clkdiv(uint slice_num, float divider);`	slice_num: slice PWM. divider: divisor aplicado al reloj.	hardware/pwm.h	Modifica la frecuencia de conteo del PWM.	`pwm_set_clkdiv(slice, 125.0f);`
`void pwm_set_enabled(uint slice_num, bool enabled);`	slice_num: slice. enabled: true para habilitar; false para detener.	hardware/pwm.h	Inicia o detiene un slice PWM.	`pwm_set_enabled(slice, true);`

Función / prototipo	Parámetros	Biblioteca	Descripción	Ejemplo de uso
`void gpio_set_irq_enabled(uint gpio, uint32_t events, bool enabled);`	gpio: pin. events: condición de interrupción. enabled: habilita o deshabilita.	pico/stdlib.h o hardware/gpio.h	Habilita eventos de interrupción asociados a un GPIO.	`gpio_set_irq_enabled(15, GPIO_IRQ_EDGE_FALL, true);`
`void gpio_set_irq_enabled_with_callback(uint gpio, uint32_t events, bool enabled, gpio_irq_callback_t callback);`	gpio: pin. events: flanco/evento. enabled: estado. callback: función que atenderá la interrupción.	pico/stdlib.h o hardware/gpio.h	Habilita una interrupción GPIO y registra una función callback.	`gpio_set_irq_enabled_with_callback(15, GPIO_IRQ_EDGE_FALL, true, &callback);`
`void irq_set_enabled(uint num, bool enabled);`	num: número/fuente de interrupción. enabled: activa o desactiva.	hardware/irq.h	Habilita o deshabilita una fuente de interrupción en el NVIC.	`irq_set_enabled(UART0_IRQ, true);`
`void irq_set_exclusive_handler(uint num, irq_handler_t handler);`	num: interrupción. handler: función manejadora.	hardware/irq.h	Asigna un manejador exclusivo a una interrupción.	`irq_set_exclusive_handler(UART0_IRQ, uart_handler);`
`uint uart_init(uart_inst_t *uart, uint baudrate);`	uart: periférico, por ejemplo uart0. baudrate: velocidad en baudios.	hardware/uart.h	Inicializa una UART y establece su velocidad.	`uart_init(uart0, 115200);`
`void uart_putc_raw(uart_inst_t *uart, char c);`	uart: instancia UART. c: carácter a transmitir.	hardware/uart.h	Envía un carácter directamente por UART.	`uart_putc_raw(uart0, 'A');`
`void uart_puts(uart_inst_t *uart, const char *s);`	uart: instancia UART. s: cadena que se desea transmitir.	hardware/uart.h	Envía una cadena de caracteres por UART.	`uart_puts(uart0, "Hola");`
`char uart_getc(uart_inst_t *uart);`	uart: instancia UART.	hardware/uart.h	Espera y recibe un carácter desde UART.	`char c = uart_getc(uart0);`
`bool uart_is_readable(uart_inst_t *uart);`	uart: instancia UART.	hardware/uart.h	Indica si existen datos disponibles para leer.	`if (uart_is_readable(uart0)) { ... }`
`uint i2c_init(i2c_inst_t *i2c, uint baudrate);`	i2c: instancia i2c0 o i2c1. baudrate: frecuencia del bus.	hardware/i2c.h	Inicializa un controlador I2C.	`i2c_init(i2c0, 100000);`
`int i2c_write_blocking(i2c_inst_t *i2c, uint8_t addr, const uint8_t *src, size_t len, bool nostop);`	i2c: instancia. addr: dirección del dispositivo. src: datos. len: bytes. nostop: evita STOP si es true.	hardware/i2c.h	Envía un bloque de datos por I2C.	`i2c_write_blocking(i2c0, 0x3C, datos, 2, false);`
`int i2c_read_blocking(i2c_inst_t *i2c, uint8_t addr, uint8_t *dst, size_t len, bool nostop);`	i2c: instancia. addr: dirección. dst: buffer destino. len: bytes. nostop: comportamiento STOP.	hardware/i2c.h	Lee un bloque de datos desde un dispositivo I2C.	`i2c_read_blocking(i2c0, 0x3C, datos, 2, false);`
`uint spi_init(spi_inst_t *spi, uint baudrate);`	spi: spi0 o spi1. baudrate: frecuencia deseada.	hardware/spi.h	Inicializa el periférico SPI.	`spi_init(spi0, 1000000);`
`int spi_write_blocking(spi_inst_t *spi, const uint8_t *src, size_t len);`	spi: instancia. src: datos a enviar. len: cantidad de bytes.	hardware/spi.h	Envía datos mediante SPI.	`spi_write_blocking(spi0, datos, 4);`
`int spi_read_blocking(spi_inst_t *spi, uint8_t repeated_tx_data, uint8_t *dst, size_t len);`	spi: instancia. repeated_tx_data: byte transmitido durante la lectura. dst: buffer receptor. len: bytes.	hardware/spi.h	Lee datos SPI mientras transmite repetidamente un byte.	`spi_read_blocking(spi0, 0x00, buffer, 4);`
`int spi_write_read_blocking(spi_inst_t *spi, const uint8_t *src, uint8_t *dst, size_t len);`	spi: instancia. src: datos TX. dst: buffer RX. len: cantidad de bytes.	hardware/spi.h	Transmite y recibe simultáneamente datos mediante SPI.	`spi_write_read_blocking(spi0, tx, rx, 4);`
`uint32_t get_rand_32(void);`	No recibe parámetros.	pico/rand.h	Obtiene un número pseudoaleatorio de 32 bits.	`uint32_t n = get_rand_32();`
`uint64_t get_rand_64(void);`	No recibe parámetros.	pico/rand.h	Obtiene un número pseudoaleatorio de 64 bits.	`uint64_t n = get_rand_64();`
`void get_rand_128(rng_128_t *rand128);`	rand128: apuntador donde se almacenará el valor de 128 bits.	pico/rand.h	Obtiene un número pseudoaleatorio de 128 bits.	`rng_128_t n; get_rand_128(&n);`
`bool add_repeating_timer_ms(int64_t delay_ms, repeating_timer_callback_t callback, void *user_data, repeating_timer_t *out);`	delay_ms: periodo. callback: función periódica. user_data: datos opcionales. out: estructura del temporizador.	pico/stdlib.h o pico/time.h	Crea un temporizador periódico.	`add_repeating_timer_ms(1000, timer_cb, NULL, &timer);`
`bool cancel_repeating_timer(repeating_timer_t *timer);`	timer: apuntador al temporizador que se desea cancelar.	pico/stdlib.h o pico/time.h	Cancela un temporizador periódico previamente creado.	`cancel_repeating_timer(&timer);`
`void stdio_init_all(void);`	No recibe parámetros.	pico/stdlib.h o pico/stdio.h	Inicializa los mecanismos de entrada/salida estándar configurados.	`stdio_init_all();`

Nota: cuando se muestra “X o Y”, cualquiera de esos encabezados puede dar acceso a la función indicada. La biblioteca específica sigue siendo útil para identificar el módulo del SDK al que pertenece.